

Getting and cleaning Data - JHU University Course 3

Hassan Reza Rezaye

2026-05-15

In this document, I represented my understanding of R programming language and data analysis. I explored several data file that needed to be cleaned and tidy for analyzing and resolving some specific questions. The questions are part of **Getting and Cleaning Data** course from John Hopkins University's **Data Science Specialization** on Coursera. As you moving on reading this document, you will face several files pertaining to various questions. You will see how I approached in steps to answering the questions by applying necessary packages and related functions and verbs which I learned in this course.

Question 1:

The American Community Survey distributes downloadable data about United States communities. Download the 2006 microdata survey about housing for the state of Idaho using `download.file()` from here:

<https://d396qusza40orc.cloudfront.net/getdata%2Fdata%2Fss06hid.csv>

and load the data into R. The code book, describing the variable names is here:

<https://d396qusza40orc.cloudfront.net/getdata%2Fdata%2FPUMSDDataDict06.pdf>

The tasks to be done:

- Create a logical vector that identifies the households on greater than 10 acres who sold more than \$10,000 worth of agriculture products.
- Assign that logical vector to the variable `agricultureLogical`.
- Apply the `which()` function like this to identify the rows of the data frame where the logical vector is TRUE. `which(agricultureLogical)`

Q1. What are the first 3 values that result? 1. 125, 238,262 | 2. 403, 756, 798 | 3. 25, 36, 45 | 4. 59, 460, 474

Step 1. a. download and save your file to the right working directory and then go explore your data file and see what anomalies you see in it that needs to be cleaned.

```
#know your working directory. if it is not the one you want the reset it.
# getwd()
# setwd("C:/")
# getwd()
# list.files()

url <- "https://d396qusza40orc.cloudfront.net/getdata%2Fdata%2Fss06hid.csv"
# download and save the file in the desired folder that is normally your wd
download.file(url, destfile = "ss06hid.csv")
```

Note:

(Optional) codes chunks bellow led by # are just for my practice and part of solving the question.

```
# dir.create("W3 Quiz", recursive = TRUE) # create a specific folder for our work
# file.rename("W3 Quiz", "W3_Quiz")      # name changing performed in working directory
# unlink("W3 Quiz", recursive = TRUE)     # delete the irrelevant folder

# moved the the file from the wd (GettingAndCleaningData) to
# file.rename("ss06hid.csv", "../W3_Quiz/ss06hid.csv")

# the a subfolder (W3_Quiz) in the wd.

# setwd("../W3_Quiz")
# getwd()
# list.files()
```

b. Read the data and assign it to a variable then take a look on the dataset to understand its structure. following it you concise the data by selecting the desired columns. The last task is to apply grouping and aggregation to answer the question.

```
UScom_housing <- read.csv("ss06hid.csv") #reading the file

# The file has many names. I avoided displaying them so that to keep document
# out of unnecessary cluttering. you can check the names "code book" mentioned above.
# names(UScom_housing)
# head(UScom_housing)

library(dplyr) # use dplyr package to use pipe "%>%" and other releted functions.

Housing <- UScom_housing
agricultureLogical <- Housing %>%
  group_by(ACR , AGS) %>%
  summarise(count = n(), .groups = "drop_last")

print(agricultureLogical)
```

```
## # A tibble: 16 x 3
## # Groups:   ACR [4]
##   ACR    AGS count
##   <int> <int> <int>
## 1     1    NA  3978
## 2     2     1   938
## 3     2     2   43
## 4     2     3   18
## 5     2     4    6
## 6     2     5    5
## 7     2     6   12
## 8     2    NA   60
## 9     3     1  217
## 10    3     2   28
## 11    3     3   23
## 12    3     4   25
## 13    3     5   30
## 14    3     6   77
## 15    3    NA   41
## 16   NA    NA  995
```

This answers the question. To find the first three values, you need to find the indices positions.

```
which(Housing$ACR == 3 & Housing$AGS == 6)
```

```
## [1] 125 238 262 470 555 568 608 643 787 808 824 849 952 955 1033
## [16] 1265 1275 1315 1388 1607 1629 1651 1856 1919 2101 2194 2403 2443 2539 2580
## [31] 2655 2680 2740 2838 2965 3131 3133 3163 3291 3370 3402 3585 3652 3852 3862
## [46] 3912 4023 4045 4107 4113 4117 4185 4198 4310 4343 4354 4448 4453 4461 4718
## [61] 4817 4835 4910 5140 5199 5236 5326 5417 5531 5574 5894 6033 6044 6089 6275
## [76] 6376 6420
```

Question 2.

Using the jpeg package read in the following picture of your instructor into R

<https://d396qusza40orc.cloudfront.net/getdata%2Fjeff.jpg>

Use the parameter native=TRUE. What are the 30th and 80th quantiles of the resulting data? (some Linux systems may produce an answer 638 different for the 30th quantile)

Answer Choices: 1. -15259150 -10575416 | 2. -16776430 -15390165 | 3. -10904118 -10575416
| 4. 10904118 -594524

```
url1 <- "https://d396qusza40orc.cloudfront.net/getdata%2Fjeff.jpg"
download.file(url1, destfile = "jeff.jpg", mode = "wb")
# after downloading and saving the file in the desired folder/WD
install.packages("jpeg", repos = "https://cloud.r-project.org")
```

```
## package 'jpeg' successfully unpacked and MD5 sums checked
##
## The downloaded binary packages are in
## C:\Users\test\AppData\Local\Temp\RtmpUPyqlC\downloaded_packages
```

```
# install the required package and get it from the library to read jpeg file
library(jpeg)
```

This answers the question.

```
imag <- readJPEG("./jeff.jpg", native = TRUE)
quantile(imag, probs = c(0.3, 0.8))
```

```
##          30%          80%
## -15259150 -10575416
```

```
as_tibble(quantile(imag, probs = c(0.3, 0.8)))
```

```
## # A tibble: 2 x 1
##       value
##       <dbl>
## 1 -15259150.
## 2 -10575416
```

```
#grid::grid.raster(imag)      # it plot the picture
```

Question 3

Load the Gross Domestic Product data for the 190 ranked countries in this data set:

<https://d396qusza40orc.cloudfront.net/getdata%2Fdata%2FGDP.csv>

Load the educational data from this data set:

https://d396qusza40orc.cloudfront.net/getdata%2Fdata%2FEDSTATS_Country.csv Tasks to be done:

- Match the data based on the country shortcode.
- How many of the IDs match?
- Sort the data frame in descending order by GDP rank (so United States is last). What is the 13th country in the resulting data frame?

Answer choices:

- 234 matches, 13th country is St. Kitts and Nevis
- 190 matches, 13th country is Spain
- 189 matches, 13th country is Spain
- 190 matches, 13th country is St. Kitts and Nevis
- 234 matches, 13th country is Spain
- 189 matches, 13th country is St. Kitts and Nevis

The data file GDP is messy. To answer the question, I cleaned and made it tidy for analysis.

Step 1. Understand your dataset and find out what needs to be modified and cleaned.

```
# Downloaded, saved the file and read the data files and then assigned to a variable

download.file("https://d396qusza40orc.cloudfront.net/getdata%2Fdata%2FGDP.csv",
              destfile = "GDP.csv")
download.file("https://d396qusza40orc.cloudfront.net/getdata%2Fdata%2FEDSTATS_Country.csv",
              destfile = "EDSTATS_Country.csv")

GDP <- read.csv("GDP.csv")
Edstats <- read.csv("EDSTATS_Country.csv")

# Since the GDP dataset is wide column wise and lengthy row wise and also messy,
# I avoided implementing and displaying output of the following verbs.
# head(GDP, n =15)
# tail(GDP, n =100)
# head(Edstats)
```

You can realise the messiness of the GDP dataset by the following implementations.

```
dim(GDP)      # return number of rows and columns
```

```
## [1] 330 10
```

```
# you can get the understanding of the overall structure of the data:
# number of column and rows, type of the file (this one is a dataframe)
# and data type of columns, first rows of the data
str(GDP)
```

```
## 'data.frame':   330 obs. of  10 variables:
## $ X                : chr  "" "" "" "" ...
## $ Gross.domestic.product.2012: chr  "" "" "Ranking" "" ...
## $ X.1               : logi  NA NA NA NA NA NA ...
## $ X.2               : chr  "" "" "Economy" "" ...
## $ X.3               : chr  "" "" "(millions of" "US dollars)" "" ...
## $ X.4               : chr  "" "" "" "" ...
## $ X.5               : logi  NA NA NA NA NA NA ...
## $ X.6               : logi  NA NA NA NA NA NA ...
## $ X.7               : logi  NA NA NA NA NA NA ...
## $ X.8               : logi  NA NA NA NA NA NA ...
```

```
names(GDP) # you see that the columns is not clear that should be varified
```

```
## [1] "X"                "Gross.domestic.product.2012"
## [3] "X.1"              "X.2"
## [5] "X.3"              "X.4"
## [7] "X.5"              "X.6"
## [9] "X.7"              "X.8"
```

```
# you want to get the specified rows from inside of the df in ascending order
GDP[(nrow(GDP)-100) : (nrow(GDP)-90), ]
```

```
##      X
## 230 LAC
## 231 MNA
## 232 SAS
## 233 SSA
## 234 HIC
## 235 EMU
## 236
## 237
## 238
## 239
## 240
##
## 230
## 231
## 232
## 233
## 234
## 235
## 236
## 237
```

```
## 238      Note: Rankings include only those economies with confirmed GDP estimates. Figures in i
## 239      a. Includes Former Spanish Sahara. b. Excludes South Sudan c. Covers mainland Tanzania only
## 240 controlled by the government of the Republic of Cyprus. e. Excludes Abkhazia and South Ossetia
##      X.1      X.2      X.3 X.4 X.5 X.6 X.7 X.8
## 230 NA      Latin America & Caribbean  5,344,028      NA NA NA NA
## 231 NA      Middle East & North Africa  1,540,807      NA NA NA NA
## 232 NA      South Asia  2,286,093      NA NA NA NA
## 233 NA      Sub-Saharan Africa  1,289,813      NA NA NA NA
## 234 NA      High income  49,717,634      NA NA NA NA
```

```
## 235 NA                      Euro area 12,192,344      NA NA NA NA
## 236 NA                      NA NA NA NA
## 237 NA                      NA NA NA NA
## 238 NA                      NA NA NA NA
## 239 NA                      NA NA NA NA
## 240 NA                      NA NA NA NA
```

From the above implementation on the GDP dataframe and its outputs, there appears to be rows and columns with empty cells and NAs and metha data at the beginning and end of the dataset. The column names are not clear that should be refined and varified correctly then the row with real values should be filtered.

step 2. Cleaning

a. Modifying Beginning of the data set:

```
GDP1 <- GDP[-c(0:2,4), ] # drop the unnecessary rows from the beginning
GDP2 <- GDP1             # assign to another variable to avoid unintended mistakes
                          # could be made to the modified dataframe

colnames(GDP2) <- as.character(GDP2[1,])
                          # set the appropriate row as column names or
                          # variables column which is now row 1
head(GDP2, n = 4)        # check if modification is done correctly
```

```
##      Ranking NA      Economy US dollars) NA NA NA NA
## 3      Ranking NA      Economy US dollars) NA NA NA NA
## 5 USA      1 NA United States 16,244,600 NA NA NA NA
## 6 CHN      2 NA      China    8,227,103 NA NA NA NA
## 7 JPN      3 NA      Japan    5,959,718 NA NA NA NA
```

```
GDP2 <- GDP2[-1, ]      # the row 1 which was set as column names is still as
                        # common row/observations that should be dropped.
head(GDP2, 4)
```

```
##      Ranking NA      Economy US dollars) NA NA NA NA
## 5 USA      1 NA United States 16,244,600 NA NA NA NA
## 6 CHN      2 NA      China    8,227,103 NA NA NA NA
## 7 JPN      3 NA      Japan    5,959,718 NA NA NA NA
## 8 DEU      4 NA      Germany   3,428,131 NA NA NA NA
```

```
# It can also be noticed that the first column is without a name
colnames(GDP2)[1] <- as.character("CountryCode") # renaming the first column
head(GDP2)
```

```
##      CountryCode Ranking NA      Economy US dollars) NA NA NA NA
## 5      USA      1 NA United States 16,244,600 NA NA NA NA
## 6      CHN      2 NA      China    8,227,103 NA NA NA NA
## 7      JPN      3 NA      Japan    5,959,718 NA NA NA NA
## 8      DEU      4 NA      Germany   3,428,131 NA NA NA NA
## 9      FRA      5 NA      France    2,612,878 NA NA NA NA
## 10     GBR      6 NA United Kingdom 2,471,784 NA NA NA NA
```

b. Modifying the entire dataset

```
nrow(is.na(GDP2))  # find number of row that contain at least one NA
```

```
## [1] 326
```

```
      # getting the specified rows of the dataset from the end part.  
      # it is to know the end of values of interest and omit  
      # irrelevant rows that come after  
GDP2[seq((nrow(GDP2)-115), (nrow(GDP2)-90)),]
```

```
##      CountryCode  
## 215          SOM  
## 216          MAF  
## 217          TCA  
## 218          VIR  
## 219          WBG  
## 220  
## 221          WLD  
## 222  
## 223          LIC  
## 224          MIC  
## 225          LMC  
## 226          UMC  
## 227          LMY  
## 228          EAP  
## 229          ECA  
## 230          LAC  
## 231          MNA  
## 232          SAS  
## 233          SSA  
## 234          HIC  
## 235          EMU  
## 236  
## 237  
## 238  
## 239  
## 240  
##  
## 215  
## 216  
## 217  
## 218  
## 219  
## 220  
## 221  
## 222  
## 223  
## 224  
## 225  
## 226  
## 227  
## 228
```

```

## 229
## 230
## 231
## 232
## 233
## 234
## 235
## 236
## 237
## 238      Note: Rankings include only those economies with confirmed GDP estimates. Figures in i
## 239      a. Includes Former Spanish Sahara. b. Excludes South Sudan c. Covers mainland Tanzania onl
## 240 controlled by the government of the Republic of Cyprus. e. Excludes Abkhazia and South Ossetia
##      NA      Economy US dollars) NA NA NA NA
## 215 NA      Somalia      .. NA NA NA NA
## 216 NA      St. Martin (French part) .. NA NA NA NA
## 217 NA      Turks and Caicos Islands .. NA NA NA NA
## 218 NA      Virgin Islands (U.S.) .. NA NA NA NA
## 219 NA      West Bank and Gaza      .. NA NA NA NA
## 220 NA      NA NA NA NA
## 221 NA      World 72,440,449 NA NA NA NA
## 222 NA      NA NA NA NA
## 223 NA      Low income 504,431 NA NA NA NA
## 224 NA      Middle income 22,249,909 NA NA NA NA
## 225 NA      Lower middle income 4,823,811 NA NA NA NA
## 226 NA      Upper middle income 17,426,690 NA NA NA NA
## 227 NA      Low & middle income 22,769,282 NA NA NA NA
## 228 NA      East Asia & Pacific 10,329,684 NA NA NA NA
## 229 NA      Europe & Central Asia 1,887,950 NA NA NA NA
## 230 NA      Latin America & Caribbean 5,344,028 NA NA NA NA
## 231 NA      Middle East & North Africa 1,540,807 NA NA NA NA
## 232 NA      South Asia 2,286,093 NA NA NA NA
## 233 NA      Sub-Saharan Africa 1,289,813 NA NA NA NA
## 234 NA      High income 49,717,634 NA NA NA NA
## 235 NA      Euro area 12,192,344 NA NA NA NA
## 236 NA      NA NA NA NA
## 237 NA      NA NA NA NA
## 238 NA      NA NA NA NA
## 239 NA      NA NA NA NA
## 240 NA      NA NA NA NA

```

*# Here, As previously noticed, there are empty spaced vells (non NAs, non real values)
in rows and few columns at the end.*

```

GDP2[GDP2 == ""] <- NA # assign empty cells it to NAs
GDP2[GDP2 == " "] <- NA # assign only spaced cells to NAs
# tail(GDP2, 99) check if modification is done

```

*# now that the empty spaced rows are filled with NAs, it can display values of those cell as NAs
the row indices of entire rows that doesn't have at least a single value in it.
which(apply(is.na(GDP2), 1, all))*

```

## 195 220 222 236 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256
## 191 216 218 232 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276

```

```
## 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272
## 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296
## 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292
## 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316
## 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312
## 317 318 319 320 321 322 323 324 325 326 327 328 329 330
## 313 314 315 316 317 318 319 320 321 322 323 324 325 326
```

```
# remove the rows particularly the end part rows that only contains NAs
## or filter the dataset by getting the entire rows which at least have
## a single value in in a row
GDP3 <- GDP2[!(apply(is.na(GDP2), 1, all)), ]
#head(GDP3)
tail(GDP3, n = 5)
```

```
##      CountryCode
## 235          EMU
## 237          <NA>
## 238          <NA>
## 239          <NA>
## 240          <NA>
```

```
##
```

```
## 235
```

```
## 237
```

```
## 238
```

```
## 239      Note: Rankings include only those economies with confirmed GDP estimates. Figures in i
```

```
## 240      a. Includes Former Spanish Sahara. b. Excludes South Sudan c. Covers mainland Tanzania only,
```

```
##      controlled by the government of the Republic of Cyprus. e. Excludes Abkhazia and South Ossetia
```

```
##      NA
```

```
## 235
```

```
## 237
```

```
## 238
```

```
## 239
```

```
## 240
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

```
##
```

c. modifying end of the data set. Though most of the rows that entirely had NAs in the end were omitted, there are still several rows that contains meta data which is not relevant to our analysis.

```
GDP4 <- GDP3[1: (nrow(GDP3) - 4), ]
# Dropping the rows containing meta data from the end of the dataframe
tail(GDP4, 5)
```

```
##      CountryCode Ranking NA      Economy US dollars)      NA NA
## 231      MNA      <NA> NA      Middle East & North Africa  1,540,807 <NA> NA NA
## 232      SAS      <NA> NA      South Asia      2,286,093 <NA> NA NA
## 233      SSA      <NA> NA      Sub-Saharan Africa  1,289,813 <NA> NA NA
## 234      HIC      <NA> NA      High income      49,717,634 <NA> NA NA
## 235      EMU      <NA> NA      Euro area      12,192,344 <NA> NA NA
##      NA NA
## 231 NA NA
## 232 NA NA
## 233 NA NA
## 234 NA NA
## 235 NA NA
```

d. Subsetting the dataset

```
# Subsetting the data frame in order to drop columns of not our interest
# reassign it to another variable for the precaution of not losing it
GDP5 <- GDP4[, -c(3, 6:10)]
```

```
head(GDP5, 3)
```

```
##   CountryCode Ranking      Economy US dollars)
## 5         USA      1 United States 16,244,600
## 6         CHN      2         China  8,227,103
## 7         JPN      3         Japan  5,959,718
```

```
dim(GDP5)
```

```
## [1] 228  4
```

```
tail(GDP5, n = 5) # Not: the data is clean and got tidy as want. it is now ready for analysis.
```

```
##   CountryCode Ranking      Economy US dollars)
## 231         MNA   <NA> Middle East & North Africa 1,540,807
## 232         SAS   <NA>          South Asia      2,286,093
## 233         SSA   <NA> Sub-Saharan Africa 1,289,813
## 234         HIC   <NA>          High income 49,717,634
## 235         EMU   <NA>          Euro area 12,192,344
```

```
write.csv(GDP5, "GDP5_clean.csv", row.names = FALSE)
```

Now the data GDP5 is clean and tidy. It is ready for analysing and solving the corresponding questions.

However, the next step is to understand the second data file EDSTATS_country.csv. Similar to GDP dataset, It also needs to be cleaned and be tidy for analyzing and resolving the question. I have already downloaded, saved and read Edstats along with GDP in the beginning.

After careful exploration, I found out that the Edstats data file is tidier than GDP data file. It has clear header and column name, no rows and column that just include entirely NAs or empty space, no meta data in it. The only thing, I had get rid of is many column/variables which are not of our interest. Accordingly, I subsetting the data.

You can better understand the file by going through the following implementations.

```
# explore and understand the Edstats file for any anomalies
```

```
str(Edstats) # the dataset has clear and refined columns
```

```
## 'data.frame':   234 obs. of  31 variables:
## $ CountryCode      : chr  "ABW" "ADO" "AFG" "AGO" ...
## $ Long.Name        : chr  "Aruba" "Principality of Andorra" "Islami
## $ Income.Group     : chr  "High income: nonOECD" "High income: nonO
## $ Region           : chr  "Latin America & Caribbean" "Europe & Cen
## $ Lending.category : chr  "" "" "IDA" "IDA" ...
## $ Other.groups     : chr  "" "" "HIPC" "" ...
```

```
## $ Currency.Unit : chr "Aruban florin" "Euro" "Afghan afghani" "
## $ Latest.population.census : chr "2000" "Register based" "1979" "1970" ...
## $ Latest.household.survey : chr "" "" "MICS, 2003" "MICS, 2001, MIS, 2006,
## $ Special.Notes : chr "" "" "Fiscal year end: March 20; reportin
## $ National.accounts.base.year : chr "1995" "" "2002/2003" "1997" ...
## $ National.accounts.reference.year : int NA NA NA NA 1996 NA NA 1996 NA NA ...
## $ System.of.National.Accounts : int NA NA NA NA 1993 NA 1993 1993 NA NA ...
## $ SNA.price.valuation : chr "" "" "VAB" "VAP" ...
## $ Alternative.conversion.factor : chr "" "" "" "1991-96" ...
## $ PPP.survey.year : int NA NA NA 2005 2005 NA 2005 2005 NA NA ...
## $ Balance.of.Payments.Manual.in.use : chr "" "" "" "BPM5" ...
## $ External.debt.Reporting.status : chr "" "" "Actual" "Actual" ...
## $ System.of.trade : chr "Special" "General" "General" "Special" .
## $ Government.Accounting.concept : chr "" "" "Consolidated" "" ...
## $ IMF.data.dissemination.standard : chr "" "" "GDDS" "GDDS" ...
## $ Source.of.most.recent.Income.and.expenditure.data: chr "" "" "" "IHS, 2000" ...
## $ Vital.registration.complete : chr "" "Yes" "" "" ...
## $ Latest.agricultural.census : chr "" "" "" "1964-65" ...
## $ Latest.industrial.data : int NA NA NA NA 2005 NA 2001 NA NA NA ...
## $ Latest.trade.data : int 2008 2006 2008 1991 2008 2008 2008 2008 NA
## $ Latest.water.withdrawal.data : int NA NA 2000 2000 2000 2005 2000 2000 NA 19
## $ X2.alpha.code : chr "AW" "AD" "AF" "AO" ...
## $ WB.2.code : chr "AW" "AD" "AF" "AO" ...
## $ Table.Name : chr "Aruba" "Andorra" "Afghanistan" "Angola"
## $ Short.Name : chr "Aruba" "Andorra" "Afghanistan" "Angola"
```

```
# since the data is wider column wise, I did not apply below functions to
# display beginning and end of the dataset.
# colnames(Edstats)
# head(Edstats)
# tail(Edstats)
```

```
# there are difference in the number of rows
dim(GDP5)
```

```
## [1] 228 4
```

```
dim(Edstats)
```

```
## [1] 234 31
```

```
print(tibble(colSums(is.na(Edstats))), n= 20) #find number of NAs in each column of the df
```

```
## # A tibble: 31 x 1
##   `colSums(is.na(Edstats))`
##   <dbl>
## 1 0
## 2 0
## 3 0
## 4 0
## 5 0
## 6 0
```

```
## 7 0
## 8 0
## 9 0
## 10 0
## 11 0
## 12 197
## 13 149
## 14 0
## 15 0
## 16 89
## 17 0
## 18 0
## 19 0
## 20 0
## # i 11 more rows
```

From what the question asked to accomplish, we need only couple of the relevant columns which CountryCode and Income.group.

```
# the files has many column of which we just need three column of our interest
Edstats_Subset <- Edstats[, c(1,3)]
head(Edstats_Subset)
```

```
## CountryCode Income.Group
## 1 ABW High income: nonOECD
## 2 ADO High income: nonOECD
## 3 AFG Low income
## 4 AGO Lower middle income
## 5 ALB Upper middle income
## 6 ARE High income: nonOECD
```

```
# the following tow syntax is just for practice and optional to better understand the columns.
# table(Edstats_Subset$CountryCode) # how many times each value repeated in a column
# find the only values that repeats more than one time
# table(Edstats_Subset$CountryCode)[table(Edstats_Subset$CountryCode) >1]
```

Hereafter, I went on applying the tasks mentioned in the question as follows.

Q_a. Match the data based on the country shortcode.

```
# Based on "CountryCode" column, It returns a number which cover matching values
# of both datasets and exclude the values that are different.
length(intersect(Edstats_Subset$CountryCode, GDP5$CountryCode))
```

```
## [1] 224
```

```
dim(GDP5) # there 225 rows
```

```
## [1] 228 4
```

```
# however we come to understand that only 190 countries are ranked and the rest
## are irrelevant to the analysis of this question
GDP5[(nrow(GDP5)-45) : (nrow(GDP5)-30), ]
```

```
##      CountryCode Ranking      Economy US dollars)
## 187      DMA      183      Dominica      480
## 188      TON      184      Tonga      472
## 189      FSM      185      Micronesia, Fed. Sts.      326
## 190      STP      186 S\oe3o Tom\oe9 and Principe      263
## 191      PLW      187      Palau      228
## 192      MHL      188      Marshall Islands      182
## 193      KIR      189      Kiribati      175
## 194      TUV      190      Tuvalu      40
## 196      ASM      <NA>      American Samoa      ..
## 197      ADO      <NA>      Andorra      ..
## 198      CYM      <NA>      Cayman Islands      ..
## 199      CHI      <NA>      Channel Islands      ..
## 200      CUW      <NA>      Cura\oe7ao      ..
## 201      DJI      <NA>      Djibouti      <NA>
## 202      FRO      <NA>      Faeroe Islands      ..
## 203      PYF      <NA>      French Polynesia      ..
```

It gives the answer to the other parts of the question:

Q_b. How many of the IDs match?

Q_C. Sort the data frame in descending order by GDP rank (so United States is last).

What is the 13th country in the resulting data frame?

```
# made some modification in GDP5 Dataframe: filtered by the 191 countries which
## are ranked and then leave out others
# we also want change Ranking column's data type from character to numeric and
## arrange it in descending order
GDP6 <- GDP5 %>% mutate(Ranking = as.numeric(Ranking)) %>%
  filter(Ranking < 191) %>%
  arrange(desc(Ranking))
#view or print(n=inf) # it prints the entire dataset
as_tibble(GDP6)
```

```
## # A tibble: 190 x 4
##   CountryCode Ranking Economy      `US dollars)`
##   <chr>      <dbl> <chr>      <chr>
## 1 TUV      190 Tuvalu      " 40 "
## 2 KIR      189 Kiribati      " 175 "
## 3 MHL      188 Marshall Islands      " 182 "
## 4 PLW      187 Palau      " 228 "
## 5 STP      186 S o Tom and Principe      " 263 "
## 6 FSM      185 Micronesia, Fed. Sts.      " 326 "
## 7 TON      184 Tonga      " 472 "
## 8 DMA      183 Dominica      " 480 "
## 9 COM      182 Comoros      " 596 "
## 10 WSM      181 Samoa      " 684 "
## # i 180 more rows
```

The following two approach gets the matching values in both dataset that is 189.

```
# this just give the number of matching values in both datasets
length(intersect(unique(Edstats_Subset$CountryCode), unique(GDP6$CountryCode)))
```

```
## [1] 189
```

```
# it first marge both data set by key column "Countrycode"
MergedData <- merge(GDP6, Edstats_Subset, by = "CountryCode")
head(MergedData)
```

```
## CountryCode Ranking      Economy US dollars)      Income.Group
## 1      ABW      161      Aruba      2,584 High income: nonOECD
## 2      AFG      105      Afghanistan 20,497      Low income
## 3      AGO      60      Angola      114,147 Lower middle income
## 4      ALB      125      Albania      12,648 Upper middle income
## 5      ARE      32 United Arab Emirates 348,595 High income: nonOECD
## 6      ARG      26      Argentina 475,502 Upper middle income
```

```
# it displays 189 rows which is also matching number of values. it give the answer ## for the first par
nrow(MergedData)
```

```
## [1] 189
```

the c part of the question is solved - 13th country is St. Kitts and Nevis

```
GDP6[13,]
```

```
## CountryCode Ranking      Economy US dollars)
## 13      KNA      178 St. Kitts and Nevis      767
```

Question 4.

What is the average GDP ranking for the High income: OECD and High income: nonOECD group?

Answer choices:

1. 23.966667, 30.91304 | 2. 23, 30 | 3. 30, 37 | 4. 32.96667, 91.91304 | 5. 23, 45 | 6. 133.72973, 32.96667

```
head(MergedData)
```

```
## CountryCode Ranking      Economy US dollars)      Income.Group
## 1      ABW      161      Aruba      2,584 High income: nonOECD
## 2      AFG      105      Afghanistan 20,497      Low income
## 3      AGO      60      Angola      114,147 Lower middle income
## 4      ALB      125      Albania      12,648 Upper middle income
## 5      ARE      32 United Arab Emirates 348,595 High income: nonOECD
## 6      ARG      26      Argentina 475,502 Upper middle income
```

It gives the answer is.

```
MergedData %>% group_by(Income.Group) %>%
  summarise(GDP.Ranking_ave = mean(Ranking, na.rm = TRUE)) %>%
  filter(Income.Group == "High income: OECD"
         | Income.Group == "High income: nonOECD")
```

```
## # A tibble: 2 x 2
##   Income.Group      GDP.Ranking_ave
##   <chr>            <dbl>
## 1 High income: OECD      33.0
## 2 High income: nonOECD   91.9
```

Question 5.

- Cut the GDP ranking into 5 separate quantile groups.
- Make a table versus Income.Group.
- How many countries are Lower middle income but among the 38 nations with highest GDP?

Answer choices: 5 | 0 | 3 | 18

```
#check if Income.group has any NAs
as_tibble(sum(is.na(MergedData$Income.Group)))
```

```
## # A tibble: 1 x 1
##   value
##   <int>
## 1     0
```

Q _ a. first part of the question: Cut the GDP ranking into 5 separate quantile groups.

```
# we apply quantile() to divide the Ranking in five equal part in quantile basis
# "prob = " is gets the comstomized quantile of a the numeric vector
Break <- quantile(MergedData$Ranking, probs = seq(0, 1, 0.2), na.rm = TRUE)
Break
```

```
##    0%    20%    40%    60%    80%   100%
##    1.0   38.6   76.2  113.8  152.4  190.0
```

```
# the Ranking column values are broke into five peaces/intervals based on
# the five intervals calculated by quantile()
# the result is then assigned to a new column in the df
MergedData$RankingQuantile <- cut(MergedData$Ranking,
                                breaks = Break)
# displly the five unique intervals or quantiles
unique(MergedData$RankingQuantile)
```

```
## [1] (152,190] (76.2,114] (38.6,76.2] (114,152] (1,38.6] <NA>
## Levels: (1,38.6] (38.6,76.2] (76.2,114] (114,152] (152,190]
```

```
# Values of the Ranking Column (GDP ranking of the countries) arranged in Descending order.
# the high ranking countries start from value 1 and goes upward until 190 which
## ends to the lowest GDP ranking country.
# it mean the answer is in first 38 ranking values which accounts for nations with the highest GDP.
head(MergedData, 15)
```

```
##      CountryCode Ranking      Economy US dollars)      Income.Group
## 1      ABW      161      Aruba      2,584 High income: nonOECD
## 2      AFG      105      Afghanistan      20,497      Low income
## 3      AGO      60      Angola      114,147      Lower middle income
## 4      ALB      125      Albania      12,648      Upper middle income
## 5      ARE      32      United Arab Emirates      348,595 High income: nonOECD
## 6      ARG      26      Argentina      475,502      Upper middle income
## 7      ARM      133      Armenia      9,951      Lower middle income
## 8      ATG      172      Antigua and Barbuda      1,134      Upper middle income
## 9      AUS      12      Australia      1,532,408      High income: OECD
## 10     AUT      27      Austria      394,708      High income: OECD
## 11     AZE      68      Azerbaijan      66,605      Upper middle income
## 12     BDI      162      Burundi      2,472      Low income
## 13     BEL      25      Belgium      483,262      High income: OECD
## 14     BEN      140      Benin      7,557      Low income
## 15     BFA      128      Burkina Faso      10,441      Low income
##      RankingQuantile
## 1      (152,190]
## 2      (76.2,114]
## 3      (38.6,76.2]
## 4      (114,152]
## 5      (1,38.6]
## 6      (1,38.6]
## 7      (114,152]
## 8      (152,190]
## 9      (1,38.6]
## 10     (1,38.6]
## 11     (38.6,76.2]
## 12     (152,190]
## 13     (1,38.6]
## 14     (114,152]
## 15     (114,152]
```

Q _ c How many countries are Lower middle income but among the 38 nations with highest GDP?

```
MergedData %>% group_by(Income.Group, RankingQuantile) %>%
  summarize(count = n(), .groups = "drop") %>%
  arrange(RankingQuantile) %>%
  filter(Income.Group == "Lower middle income") %>%
  as_tibble()
```

```
## # A tibble: 5 x 3
##   Income.Group      RankingQuantile count
##   <chr>           <fct>           <int>
## 1 Lower middle income (1,38.6]           5
```

```
## 2 Lower middle income (38.6,76.2]      13
## 3 Lower middle income (76.2,114]       11
## 4 Lower middle income (114,152]        9
## 5 Lower middle income (152,190]       16
```

```
# therefor the answer should be in the interval (1,38.6]
MergedData %>% group_by(Income.Group, RankingQuantile) %>%
  summarize(Ranking_count = n(), .groups = "drop") %>%
  filter(Income.Group == "Lower middle income" & RankingQuantile == "(1,38.6]")
```

```
## # A tibble: 1 x 3
##   Income.Group      RankingQuantile Ranking_count
##   <chr>             <fct>                <int>
## 1 Lower middle income (1,38.6]                5
```